

print: 값을 구하는 등의 상황에서 실행하는 가장 기본적인 내장 함수

int: 정수형

float: 실수형

str: 문자형(+문자열)

bool: 논리형(True, False)

input: 데이터 입력(사용자가 입력하기 전까지, 대기 상태를 유지)

type casting: 자료형 변환

f-string: 복잡하게 만들어지는 것을 방지하기 위한 방법

(에러, 오타를 줄일 수 있는 가장 최선책)

format(): 파이썬의 문자열을 보다 깔끔하기 위해 '{}'를 사용하는 것

코드 컨벤션: 사람이 읽기 용이하게 만든 규칙 [매우 중요]

연산자: 산술, 할당, 비교. 논리 연산자로 구분(논리연산자: and, or, not)

산술 연산자: 사칙연산자와 컴퓨터 프로그램에서만 사용하는 나머지, 몫, 지수

연산자를 뜻한다 (+, -, *, /), (%(나머지), //(몫), **(거듭제곱))

할당 연산자(대입 연산자): 변수에 값을 대입하는 데 사용하는 연산자

복합대입 연산자: "+=", "-=", "*=", "/=", "%=", "//=", "**="

비교 연산자: "==, !=, >, >=, <, <="

논리 연산자: 피연산자의 논리 자료형(True/False)을 이용하는 연산자

(and: 그리고, or: 또는, not: '부정'(True면 False로 / False면 True로 출력))

모듈: 특정 기능을 모아놓은 파일(먼저, import를 이용해서 모듈은 가져와야 함)

operator 모듈(+, -, *, / 등의 연산자 모듈)

(모듈을 사용하면 연산자 기호를 사용하는 것보다 더욱 빠른 실행이 가능하다)

날짜 관련 모듈: datetime

-> from datetime import datetime (다른 모듈과 달리, 'from datetime'을 사용함)

현재 '일' 구하기 ex: print(datetime.today().day)

shuffle 모듈: import random - shuffle

if: 만약에~ '1번 조건'이라면 (ex: if num1 > 10:)

elif: 만약에~ '2번 조건'이면

else: '1번 조건'/'2번 조건'이 아니라면 / 그렇지 않다면~

if~else: 양자택일의 상황에만 사용할 것

if~elif: 다중선택의 상황에 사용할 것

if~elif~else:

1순서: if(맨 처음만) --> 2순서: elif(N번 반복 가능) --> 3순서: else(무조건 마지막만)

if~elif~else: <-(컴플릿)

논리상 'A-B-C-D-E-F'의 순서를 'A-B-D-C-E-F'로 변경하는 것이 가능하지만,

원래는 'C'에 해당/인식되는 데이터가 'D'로 해당/인식되는 결과가 발생함

상수: 절대변하지 않는 수, 또는 문자, 데이터 등

python에서는 자체 지원하지 않기 때문에 [대문자 + 스네이크 표기법]을

활용해 구분함

리팩터링: 결과는 똑같지만, 코드 내부를 깨끗하고, 유지보수가 쉽게 만드는 것

(만들어진 코드를 다시 읽고, 이해하여 처음 만든 프레임 워크 등에 붙이거나,

작성한 코드들을 모듈이나 상수, 함수 등으로 변경해 실행 파일(main) 등을

수정함)

중첩 조건문: 조건문 내에 또 다른 조건문을 작성한 조건문

삼항 연산자를 활용한 if~else문

(1항: '참'일때 값 | [if] 2항: 조건식(+ >=) | [else] 3항: '거짓'일때 값)

반복문: for문, while문

for문: ~하는 동안(지정된 횟수에 따라 반복 -> 횟수 반복)

```
# range: 범위지정 함수
# range(0, 10, 1): (=0부터, 9까지, 1씩 -> "(시작의 값, 끝의 값, 단계의 단위)")
# (for문의 범위를 지정-> for 변수 in range():)
```

```
# for i in range() -> i: iterable(반복 가능한)의 데이터를 순차적으로 저장하는 변수
# ex: range(11)에서 시작의 값은 0, 끝의 값은 10(=11-1), 단계의 단위는 1씩 증가
```

```
# 문자열 활용한 for문: iterable에는 문자열도 가능함
```

```
# while문: ~하는 동안(특정조건에 의한 반복 -> 조건 반복)
# (조건식의 결과가 참인 경우, 실행문을 계속해서 반복함)
```

```
# 반복문 내 실행 제어 (continue: 계속해서 반복 실행, break: 실행 중단, 탈출)
```

```
# pass 키워드: if문, for문, while문, 함수 등의 실행에 미완성, 수정이 필요한
# 코드 등으로 'error'가 발생해 실행 결과를 얻지 못하는 것을 방지하기 위함
```

```
# 컨테이너 자료형: 많은 데이터를 효율적으로 관리하기 위해 사용함
# 컨테이너 자료형 종류: 리스트(list), 튜플(tuple), 딕셔너리(dictionary)
```

```
# 기초 데이터 타입: 변수명 선언과 초기화에 따른 데이터 입력 및 삽입
# 참조 데이터 타입(= 레퍼런스 데이터 타입): 해당되는 컨테이너 자료형인
# 리스트와 딕셔너리의 변수명은 데이터를 직접 담지 않고, 데이터 모음에서
# 첫번째 데이터 주소를 가진다 -> 데이터를 '참조'한다)
# (가리키는/참조하는 데이터가 변경되면, 변경된 데이터의 주소를 가리키며,
# 원래 참조한 주소의 데이터 들은 더미 데이터가 된다)
```

```
# list: 리스트를 선언할때는 대괄호를 이용해서 데이터를 묶고, 쉼표로 구분한다
# 리스트에 포함되는 데이터는 어떤 자료형이든 상관없이 하나의 리스트로 담김
# (하나의 리스트에 다양한 데이터 타입이 담기는 것은 python과 javascript뿐임)
```

```
# how to 리스트, 특정 아이템 조회
```

```
# 인덱스(index): 컨테이너 자료형 등의 데이터 모음에 데이터가 저장원때마다
```

```
# 자동적으로 부여되는 순서
# (인덱스 순서는 기본적으로 0부터 시작하며, 데이터를 불러올때 인덱스 번호를
# 참조해 불러온다)
```

```
# 인덱스 에러: 만약 컨테이너 자료형에 존재하지 않는 데이터의 인덱스를
# 참조할 때, 발생하는 에러
```

```
# 리스트 길이(= 아이템 개수) 조회 방법
# len(): '리스트 길이 = 리스트에 속한 아이템의 개수'를 확인하는 함수
```

```
# (len() 함수는 문자열의 길이 또한 조회 가능하다)
# 문자열(str)은 문자(character)를 나열한 리스트와 같다
# -> 문자열 또한 len() 함수로 길이를 조회할 수 있다.
```

```
# 특정 위치에 아이템 삽입: .insert()
# (리스트에서 원하는 위치 아이템을 삽입할 때 사용함)
```

```
# dynamic data management: 리스트의 데이터 메모리를 여유롭게 관리하는 방법
# (insert 함수로 데이터가 추가될 때, 다른 리스트의 데이터 메모리에 영향을
# 끼칠 수 없기 때문에 데이터가 추가된 리스트의 메모리는 전부 옮겨진다)
# -> 그래서 이러한 상황을 예방하기 위해 데이터 메모리의 관리는 항상 여유롭게
# 유지/관리된다
```

```
# 리스트에 또 다른 리스트를 연결할 때: .extend()
```

```
# 리스트에서 아이템 삭제
# .pop(n): 리스트의 아이템을 삭제(기본적으로 '()'에 아무것도 없으면, 마지막)
# del: 'del 리스트명[N]'-> 리스트의 N번째의 특정 아이템 삭제
# .remove('~'): 리스트에서 아이템의 위치를 모르는 상황에 특정 아이템을 삭제
```

```
# .pop()와 del은 인덱스 번호를 기준으로 하고, .remove()는 데이터명을 기준함
# .pop()은 삭제한 값을 반환하지만, del과 .remove()는 반환하지 않음
```

```
# 리스트의 데이터 정렬
```

```
# .sort(): 함수는 리스트의 아이템을 정렬하는데 사용됨
```

```
# -> .sort() / .sort(reverse = False): 오름차순(ASC)
# -> .sort(reverse = True): 내림차순(DESC)
# (문자열, 한글 데이터 또한 정렬 가능함)
```

```
# 리스트 데이터 뒤집기
# .reverse(): 리스트에 속한 데이터들을 역순으로 뒤집는 함수
```

```
# 리스트 슬라이싱: 리스트에서 필요한 부분의 아이템만 뽑아내는 것
```

```
# [n:m] : n 인덱스부터 (m-1) 인덱스 까지의 아이템을 슬라이싱(추출)한다
```

```
# 리스트(list) == 튜플(tuple)
```

```
# tuple(튜플): 튜플은 리스트와 다르게 소속된 아이템의 변경, 추가, 삭제 등의
# 수정행위가 불가능하고, 오직 '조회'만 가능하다(처음 데이터가 정해지면, 불변)
# 데이터의 수정을 막아야 하는 경우에 '튜플'이 사용
```

```
# (->튜플은 리스트와 다르게 []가 아닌, ()를 사용해 아이템을 선언한다)
```

```
# 튜플 조회
# 튜플은 아이템의 수정이 불가능하기 때문에 아이템의 삽입, 삭제, 정렬 등의
# 기능은 없고, 조회하는 기능을 주로 사용한다
```

```
# 특정 아이템의 인덱스(index) 조회
# .split(쪼갬다)
```

```
# 튜플 안의 아이템 유/무 확인하기
# in과 not in을 사용하면 튜플 안에 특정 아이템의 존재 유/무를 확인할 수 있다
```

```
# 튜플과 튜플의 결합(= 리스트와 리스트의 결합과 같음)
```

```
# 얕은 복사
# 깊은 복사
```

```
# copy (=깊은 복사 / 데이터 연산 속도가 훨씬 빠름)
```

```
# import copy
# 변수명 = copy.deepcopy(numA)
```

```
# 튜플 슬라이싱: tuple slicing(=리스트 슬라이싱과 동일)
```

```
# 리스트와 튜플 간 변환(형변화, casting)
```

```
# 불가피하게 튜플의 아이템을 수정하려면 리스트로 변환해야 한다
# 또한 리스트로 선언된 데이터를 수정이 안 되도록 하려면 변환해야 한다
# 다음은 데이터 변환을 통해 리스트와
```

```
# 딕셔너리(dictionary): 리스트, 튜플과 같이 파이썬에서 많이 사용되는 컨테이너
# 자료형으로 리스트가 인덱스를 이용해 아이템을 참조한다면, 딕셔너리는 인덱스
# 대신에 '키(key)'를 이용해 아이템을 참조한다
```

```
# 딕셔너리는 '키(key)'와 '값(value)'로 이루어져 있고, 이를 '키 값'이라고 한다
# --> '키-> 키값 : k' / '값-> 밸류값 : v'
```

```
# 딕셔너리 정의
# 딕셔너리(형태) -> {
```

```
# }
```

```
1차원: nums[]
2차원: nums[][]
3차원: nums[][][]
4차원: nums[][][][]
N차원: nums[][][][] --- []
-데이터 구조 설계의 권장 차원: 2차원 까지만
```

딕셔너리 조회/삽입/수정/삭제

딕셔너리 '조회/삽입/수정/삭제'를 CRUD라고 한다
CRUD는 Create, Read, Update, Delete를 말한다

즉, 데이터를 생성(Create), 조회(Read), 수정(Update), 삭제(Delete) 하는 것을
말한다

그렇다면, 딕셔너리에서 CRUD는 딕셔너리 컨테이너 자료형에
데이터를 추가(Create), 조회(Read), 수정(Update), 삭제(Delete) 하는 것을
말한다

CRUD는 프로그래밍 뿐만 아니라 데이터베이스에서도 사용되는 용어이다

(딕셔너리 내부의 키값과 밸류값은 튜플로 구성되어 있다)

함수(function)

python에서는 '함수'가 핵심
java에서는 '클래스'가 핵심

함수란, 특정 수식에 x값을 대입하면 y값이 정해지는 것으로 "y=f(x)" 라고 한다
함수의 구조: $y = 3x + 5$ 의 수식에 x값을 '2'을 대입하면, y값은 '11'로 정해진다

프로그래밍의 함수 또한 수학의 함수와 동일하게 값을 넣어주면 특정
기능을 수행한 연산 결과를 출력한다. 여기서 특정 기능이란 덧셈 같은
비교적 간단한 연산부터 네트워크 연결, 회원 인증, 메일 발송과 같이
복잡하고 어려운 작업까지 모두 포함한다.

복잡한 함수 ex)
네트워크 주소 => 네트워크 연결 가능 => 연결 결과
아이디, 비밀번호 => 회원 인증 가능 => 인증 결과

함수 종류
1. 내장 함수: 파이썬에서 기본적으로 제공하는 함수
ex) print(), input() 등
2. 사용자 함수: 사용자가 자신의 입장의 맞게 만드는 함수

코드(기능) 재사용 -> 함수
데이터 재사용 -> 변수

모듈: 특정 기능의 작은 프로그램을 뜻한다.
모듈화: 함수를 사용한다는 것은 특정 기능을 다른 프로그램에 쉽게
이식하여 사용할 수 있고, 시간도 단축됨

함수 정의하기: 사용자 함수를 만든다는 것
함수를 정의할 때, 'def' 키워드를 사용한다. 그리고 함수명 콜론(:), 실행부를
이용한다.

함수 정의부/선언부: def 함수명():
-> if문, for문, while문, 다른 함수등을 실행

함수 호출부
다른 함수, if문, for문, while문 그리고 main파일에서 호출하여 실행
(함수 내에서 또 다른 함수 호출: 함수 안에서 또 다른 함수를 호출하는 것)

함수명 규칙
1. 내장 함수명과 동일하면 안된다.
2. 첫 글자는 주로 소문자를 사용하자.
3. 첫 글자는 숫자로 시작할 수 없다. ex) 1greet(): x, greet1(): o
4. 특수문자를 사용할 경우에는 _(언더바)만 사용하자.
(그 이외의 특수문자는 사용금지)
5. 2개 이상의 단어가 조합되는 경우, 스네이크 또는 카멜 표기법을 사용하자.
(스네이크: send_message(), 카멜: sendMessage())

함수를 여러번 선언하면, 선언한 만큼 실행된다.
함수를 선언하고, 사용하지 않으면 메모리만 차지해 메모리가 낭비된다

파이썬에서만, 함수 안에서나 바깥에서 변수를 선언하거나 데이터를
입력받아도 함수 실행에 있어서 문제가 발생하지 않음.

재귀 함수: 정의 단계에서 자기 자신을 다시 호출하여 문제를 해결하는 함수

(함수는 변수와 달리, 메모리 상에서 무한 복사 및 복제가 되면서 메모리가
full되면서 셧다운되기 전에 시스템이 중단한다:
'[Previous line repeated 996 more times]' 996번 이상 실행 불가하다는 것)

('A'함수를 실행할 때, 1번 실행하고 안에 있는 'A'함수를 다시 불러와서
메모리의 다른 곳에 다시 'A'함수를 가리키는 호출부를 만들고 이게 계속되는
상태이다)

1개의 함수에는 1개의 기능을 담을 것(1개 함수에 2개 이상의 기능을 담는
것은 지양할 것)
최대한 가능한 만큼 부품화(부분 모듈화) 하는게 이상적이다

TDD(Test-Driven-Development): 개발방법론(테스트 주도 개발)
실제 코드를 개발하기 전에, 테스트 케이스를 먼저 작성하는 소프트웨어
개발 방법론
[실패하는 테스트 작성(Red) -> 테스트 통과 토드 작성(Green)
-> 리팩토링(Blue)]의 짧은 주기를 반복해 고품질의 버그 없는 코드를
완성하는 방식

지역변수 & 전역변수
변수의 범위(scope)에 따라 구분됨

지역변수(Local Variable)
1. 선언된(만들어진) 함수, 블록 안에서만 유효하며, 함수 바깥의 외부에서는
해당 변수에 대한 접근이 불가능함
2. 프로그램 실행 중 해당 블록이나 함수가 호출될 때, 메모리에 생성되고,
작업이 종료하면 즉시 사라진다

전역변수(Global Variable)
1. 프로그램이 시작될때 생성되고 종료할때 소멸한다
2. 코드 내의 모든 함수나 변수에서 별도의 전달 등이 없어도 자유롭게
공유가 가능함

지역변수 and 전역변수

함수 내에 지역변수와 함수 바깥의 전역변수가 동시에 존재할때, 지역변수를
우선 순위로서 사용한다

global 키워드
함수 내부에서 전역변수를 수정할 때, global 키워드를 사용한다
기본적으로 함수 내부에 값을 대입하면, 파이썬에서는 해당 변수를
'지역변수'로서 인식하고 error가 발생한다(전역변수에서 찾는 시도도 없음)

(일반적인 전역변수는 불변성을 가지지만, 딕셔너리와 함수 내부의 지역
변수의 경우에는 가변성을 가진다)
(딕셔너리는 가변성을 가진 객체이기 때문에 함수 내부에서도 해당되는
변수의 값이 변경된다)

(레퍼런스 타입 객체(가변 객체 / Mutable): dictionary(딕셔너리), list(리스트))
(일반 타입 객체(불변 객체 / Immutable): int, float, str, bool, tuple(튜플))

딕셔너리
딕셔너리란, 파이썬에서 묶음 데이터를 관리하기 위한 컨테이너 자료형 중
하나이다
키(key): 밸류(value)로 데이터를 관리한다, 이 말은 딕셔너리에는 인덱스가
존재하지 않는다

매개변수
매개: 둘 사이에서 양 편의 '관계를 맺어'주는 것
함수를 사용하기 위해 먼저 함수를 정의하고 필요할 때 호출하는데,
이 때 함수를 정의하는 쪽을 함수(정의부), 함수를 호출하는 쪽을
호출부라고 한다

함수를 호출할 때, 데이터를 넘겨줄 수 있는데 이 데이터를 '인수'라고 한다
함수 정의부는 인수를 받으면, '매개변수'라는 변수에 저장한다
-> 매개변수는 지역변수의 일종이다

studentScores라는 전역변수를 while문에서 수정했음에도 오류가 발생하지
않은 이유:
-> 'studentScores'라는 변수가 레퍼런스 타입의 '리스트'이기 때문에

```
# --> 리스트 같은 레퍼런스 타입은 데이터를 직접 갖고있지 않고, 리스트
# 내부의 첫번째 데이터의 주소 값을 가지고 있기 때문에 변경/수정되어도
# 오류가 발생하지 않는다
```

```
# flag변수 vs while문의 break
# flag변수의 경우, 해당되는 함수나 for문, if문, while문의 바깥에서도
# 컨트롤이 가능하지만, while문의 break의 경우에는 해당되는 함수나
# for문, if문, while문의 내부에서만 컨트롤이 가능하기 때문에 'flag'변수가
# break보다 더 깔끔하게 기능한다
```

```
# ('매개변수=인수'의 형식으로 작성한다면, 매개변수와 인수의 인덱스번호가
# 다르더라도 원래의 목적에 따르는 방향으로, 매개변수와 인수간 정확한 관계로
# 이어진다)
# But, 매개변수의 순서에 맞게 인수를 작성하는 것을 우선해서 엄격해야 한다
```

```
# 변수명: 명사와 명사 조합 위주로 / 함수명: 작동하는 기능을 알수 있게
# (동사+명사 조합으로)
```

```
# 데이터 반환(return): 함수는 실행이 끝난 후에 결과물(값)을 호출부로
# 반환할 수 있다 -> 이때 사용하는 키워드가 'return'이다
```

```
# 모듈이란?
# 프로그램을 개발할 때, 이미 만들어져 있는 기능을 가져다 쓸 수 있는 기능
# 특정 기능(함수)을 포함하고 있는 파일(xxx.py)로 다른 프로그램에 이식해서
# 사용하는 것
```

```
# 모듈의 장점: 프로그램 개발시간 단축, 검증된 코드로서 오류가 적음,
# 기능 구현을 분업화하고 공유할 수 있어 전체적인 작업속도를 향상시킴
# But, 위의 모듈의 장점은 단점이 될 수도 있기에 주의가 필요함
```

```
# 모듈명 단축하기: as
```

```
# ex) 유명한 모듈(명)
# import numpy --> import numpy as np
```

```
# import pandas --> import pandas as pd
```

```
# 모듈 별로 기능 테스트하는 방법
# if __name__ == '__main__':
#     fun()
```

```
# 해당: module01.py 파일이 실행 파일일때, 해당 파일의 타입은 __main__이
# 되면서 해당 명령이 실행되지만, 다른 파일에서 실행한다면, 위의 명령은
# 실행되지 않는다.
```

```
# 자주 사용하는 모듈: 내장 모듈
# 이러한 내장 모듈은 절대로 외우는 것이 아니고, 가볍게 훑고
# 필요할 때, 찾아서 사용하면 됨
```

```
# math 모듈: 수학과 관련된 모듈
# random 모듈: 난수를 발생시키는 모듈
```

```
# 파이썬 내장 함수 중에서 수학과 관련된 함수
```

```
# sum(): 전체값을 반환
# max(): 최댓값을 반환
# min(): 최솟값을 반환
# abs(): 절댓값을 반환
# pow(): 거듭제곱을 반환
# round(): 반올림한 값을 반환
```

```
# time 모듈: 시간 관련 모듈(UTC)
```

```
# localtime(): 시스템의 현재 시간을 반환
# gmtime(): GMT 시간을 반환
# strftime(): 시간 포맷 코드에 따른 출력
```

```
# setter: 세팅하는 함수(= set + userNums/uNumbers)
# getter: 데이터를 조회하는 함수(= get + userNums/uNumbers)
```

객체 지향 프로그래밍: OOP(Object-Oriented-Programming)

컴퓨터 프로그램을 명령어의 단순한 나열(:절차 지향 프로그래밍)로
보는 대신, '객체'라는 독립된 작은 단위들의 모임으로 파악하고
객체들이 서로 상호작용하도록 만드는 개발 방법론
-> 현실 세계의 사물이나 개념을 소프트웨어 안으로 녹여내기 위해
고안된 방식

객체(object): 세상에 존재하는 사람, 사물 등의 모든 형태가 있는
것이며, 속성과 기능으로 구성된다 (사물ex: 키보드, 모니터 등등)
속성(attribute): 객체를 구성하는 요소(ex: 크기, 무게, 부피, 색상 등등)
기능(function): 객체가 하는 행위(ex: 걷는다, 말한다, 클릭한다 등등)

객체(object) = 속성(attribute) + 기능(function)

객체 + 객체
객체는 단독으로도 사용되기도 하지만, 서로 유기적으로 관계를 맺고
사용되기도 한다
계산 프로그램에서 '계산기'라는 객체와 '프린트'객체가 서로
유기적으로 관계를 맺어서 입력받은 숫자를 계산하고,
프린트되는 방식으로 결과가 이루어진다

객체 지향 프로그래밍: 우리가 만드는 프로그램을 객체 단위로
구분하고, 각각의 객체를 서로 연관지어서 부품을
조립하여서 유기적인 동작을 하도록 하는 프로그래밍 패러다임이다

클래스(Class): 객체를 만들기 위한 틀/설계도
ex) 봉어빵 틀(=클래스) -> 봉어빵(=객체)

동일한 클래스를 통해 만들어진 객체라도 서로 완전히 같을 수
없으며, 전혀 다르게 독립적으로 인식된다

클래스의 구조/문법(객체를 만들기 위한 틀/설계도)

-> class선언
class FishBread: (class키워드 + class이름:)

클래스 이름은 함수와 구분하기 위해 소문자가 아닌, 대문자로 시작

-> 클래스 속성 정의
def __init__(self, f, b):
self.flour = f
self.bean = b
-> '__init__': '변수 선언 및 초기화'와 같이 메모리를 청소해
메모리를 사용할 준비를 하는 과정(=클래스 선언)
self 키워드는 클래스 자신을 가리킨다는 의미를 지님
(-> 첫번째 매개변수는 무조건 self)

-> 클래스 기능
def makeFishBread(self):
print('봉어빵 제조')
함수 'makeFishBread()'또한 self를 사용해야 'FishBread'클래스
내부에서 기능한다

텍스트 파일 다루기
문자열의 영구적으로 보관하기 위함

1단계: 파일 열기
파일을 여는 단계, 'open()' 함수를 이용한다
(파일 외 ex: 데이터 베이스 등등)
파일 열기에 성공하면, 파일은 객체로 만들어져 메모리에 생성된다

2단계: 파일 쓰기/읽기
문자열을 쓰거나 읽음, 문자열을 쓸때는 'write()' 함수를,
읽을 때는 'read()' 함수를 이용한다

3단계: 파일 닫기
파일을 닫는 단계로, 쓰거나 읽기가 끝난 파일은 'close()'
함수를 이용해 닫는다

file = open('C:/lyh/python/test.txt', 'w')
파일을 '쓰기'모드로 open한다
open('C:/lyh/python: 원하는 설치 경로/test.txt: 파일명',
'w: 파일 목적'): open()함수
result = file.write('Hello, Python.') # 쓰기(write)

```
# print(f'result: {result}')      # 쓰여진 문자열의 길이
# file.close()                   # 파일 닫기(close, 외부자원 해제)
```

```
# 예외 처리(보험)
# 세상에 모든 프로그램은 100% 완벽할 수 없다, 때문에 -> '보험'
```

```
# 예외 처리의 기본 문법
# try:
#     pass                    # 출력(x): 에러가 발생하더라도, 넘어감
# except:
#     pass                    # #Exception: division by zero
# else:
#     pass                    # try와 반대의 결과 도출
# finally:
#     pass                    # 에러가 발생여부와 관계없이 실행되는 코드
```

```
# try ~ except에 많은 코드를 넣으면, 오류가 발생한 다음의 정상적인
# 코드들은 실행되지 않는다
```

20260526 다음날부터, 시작